

Lección 12: Quality Gates with Husky

Prevención Automática de Problemas

Agenda

- Quality Gates Concept
 - Pre-commit vs Pre-push
 - Configuración con Husky
 - Ejemplos Prácticos
 - Métricas de Éxito
-

El Problema

✖ Sin Quality Gates:

1. Escribir código rápido
2. `git commit -m "fix"`
3. `git push`
4. CI falla 🚨
5. Arreglar en pánico
6. Repetir ciclo

Resultado: Tiempo perdido + stress + bugs

✅ Con Quality Gates:

1. Escribir código
2. `git commit` → checks (90s)

3. Arreglar inmediatamente
4. git push → checks (3 min)
5. CI pasa 

Resultado: Calidad + menos estrés

¿Qué son Git Hooks?

Git Hooks: Scripts que se ejecutan automáticamente en eventos de Git

```
# Eventos comunes:
pre-commit      # Antes de crear commit
pre-push        # Antes de push al remoto
commit-msg      # Al escribir mensaje de commit
```

Problema:

- Configuración manual compleja
- Difícil compartir con equipo
- No se versionan fácilmente con Git

Solución: Husky

¿Qué es Husky?

Husky: Herramienta que simplifica la configuración de Git hooks

```
# Sin Husky (difícil)
.git/hooks/pre-commit # No versionado, manual para cada dev

# Con Husky (fácil)
.husky/pre-commit     # Versionado en repo, automático para todos
```

Beneficios:

- Hooks versionados en el repositorio
- Instalación automática para todo el equipo

- Sintaxis simple y legible
 - Integración con npm/pnpm scripts
-

¿Qué son Quality Gates?

Definition:

Controles automáticos que bloquean acciones si el código no cumple estándares mínimos

Ejemplo:

```
$ git commit -m "new feature"
```

```
🔍 Running Quality Gates...  
📝 Checking code style...  
❌ ESLint found 5 errors  
🚫 COMMIT BLOCKED
```

Fix these errors first:

- Unused variable `'data'` in line 23
 - Missing semicolon in line 45
-

Arquitectura de Gates

⚡ Pre-Commit (Rápidos: 30-120s):

Checks rápidos

1. pnpm run lint # 10-20s
2. pnpm run test # 30-60s
3. pnpm run build # 20-40s

Por qué aquí:

- Rápidos, developer puede esperar
- Errores fáciles de arreglar
- Previenen 80% de problemas

🐌 Pre-Push (Lentos: 2-5 min):

```
# Checks lentos
4. pnpm run test:coverage # 60-120s
5. pnpm run test:e2e      # 120-180s
```

Por qué aquí:

- Lentos, menos frecuentes
- Developer puede hacer otra cosa
- Detectan problemas complejos

Pre-Commit Gate (1/2)

Crear `.husky/pre-commit`:

```
#!/usr/bin/env sh

echo "🔍 Running Quality Gates (pre-commit)..."

# Gate 1: Linting
echo "📝 Checking code style..."
pnpm run lint || {
  echo "❌ ESLint failed! Fix errors before committing."
  exit 1
}

# Gate 2: Unit Tests
echo "🏃 Running unit tests..."
pnpm run test -- --run || {
  echo "❌ Tests failed! All must pass."
  exit 1
}
```

Pre-Commit Gate (2/2)

```
# Gate 3: TypeScript Build
echo "🔧 Checking TypeScript..."
pnpm run build || {
  echo "❌ Build failed! Fix type errors."
  exit 1
}
```

```
echo "✅ All pre-commit gates passed!"
```

Pre-Push Gate

Crear `.husky/pre-push` :

```
#!/usr/bin/env sh

echo "🚀 Running Quality Gates (pre-push)..."

# Gate 4: Coverage Check
echo "📊 Checking test coverage..."
pnpm run test:coverage || {
  echo "❌ Coverage not met! Improve tests."
  exit 1
}

# Gate 5: E2E Tests
echo "🧑🏻 Running E2E tests..."
pnpm run test:e2e || {
  echo "❌ E2E failed! User journeys must work."
  exit 1
}

echo "✅ All pre-push gates passed! Safe to push."
```

Dar Permisos

```
chmod +x .husky/pre-commit .husky/pre-push
```

Ejemplo 1: Error de Linting

```
$ git commit -m "add shopping cart"
```

```
🔍 Running Quality Gates (pre-commit)...
```

 Checking code style...

❌ ESLint failed!
/src/components/CartItem.tsx
15:10 error 'unused' is defined but never used

BLOQUEADO hasta arreglar

Solución:

1. Ver error específico
2. Abrir CartItem.tsx línea 15
3. Quitar variable 'unused'
4. Intentar commit de nuevo

Ejemplo 2: Tests Fallando

\$ git push

 Running Quality Gates (pre-push)...
 Running E2E tests...

❌ E2E failed!

1 failed
[chromium] > checkout.spec.ts:15:3
Error: Button "Complete Order" not found

BLOQUEADO hasta arreglar

Solución:

1. Revisar error E2E
 2. Botón "Complete Order" no existe
 3. Verificar checkout component
 4. Arreglar botón o test
 5. Push de nuevo
-

Casos de Uso

Caso 1: Developer Nuevo:

Sin Gates:

- 15 commits con errores
- 3 días frustración

Con Gates:

- Primer commit bloqueado
- Aprende inmediatamente
- Commits de calidad

Caso 2: Refactoring Grande:

Sin Gates:

- Push → CI explota → 2h debugging

Con Gates:

- Tests fallan en pre-push
- Arreglar antes push
- CI pasa

Comandos Útiles

Testing Manual:

```
# Probar sin hacer commit real  
./husky/pre-commit
```

```
# Probar sin hacer push real  
./husky/pre-push
```

```
# Ver qué checks están  
cat husky/pre-commit
```

Debugging:

```
# Correr checks individuales
```

```
pnpm run lint
pnpm run test
pnpm run build
pnpm run test:coverage
pnpm run test:e2e
```

Emergency Override

⚠️ **SOLO para emergencias:**

```
# Bypass gates (emergency only!)
git commit --no-verify -m "emergency fix"
git push --no-verify
```

SIEMPRE arreglar después:

1. Crear branch separado
 2. Arreglar problemas
 3. PR limpio
-

Troubleshooting

Problema 1: Permission denied

```
chmod +x .husky/pre-commit .husky/pre-push
```

Problema 2: husky command not found

```
pnpm add -D husky
pnpm exec husky init
```

Problema 3: Gates muy lentos

```
# Mover tests lentos a pre-push
```

- ✅ Pre-commit: lint + tests + build (< 2 min)
- ✅ Pre-push: coverage + E2E (< 5 min)

Métricas de Éxito

Sin Quality Gates (2 semanas):

- Commits con errores: 45%
- Tiempo en CI fixes: 8 horas
- Bugs a staging: 12
- Code review time: 3h promedio

Con Quality Gates (2 semanas):

- Commits con errores: 5%
- Tiempo en CI fixes: 1 hora
- Bugs a staging: 2
- Code review time: 1h promedio

Mejora: 90% menos problemas, 75% menos tiempo

Configuration Setup

package.json scripts (ya existentes):

```
{
  "scripts": {
    "lint": "eslint .",
    "test": "vitest",
    "build": "tsc -b && vite build",
    "test:coverage": "vitest run --coverage",
    "test:e2e": "playwright test"
  }
}
```

Solo necesitas crear los archivos `.husky/pre-commit` y `.husky/pre-push`

Ejercicio 1: Setup Pre-commit Hook

Prompt:

Actúa como un DevOps engineer implementando Quality Gates automáticos con H

CONTEXTO: Quality Gates (puertas de calidad) son checks automáticos que blo código malo ANTES de commit/push. Sin gates: código roto llega a CI → falla fix en pánico → ciclo repetido (tiempo perdido + stress). Con gates: checks locales (90s) detectan problemas inmediatamente. Husky: herramienta que eje scripts en Git hooks (pre-commit, pre-push). Pre-commit hook: ejecuta ANTES crear commit, ideal para checks rápidos (lint, formatting). Git hooks son scripts en carpeta .husky/ que Git ejecuta automáticamente.

TAREA: Configura pre-commit hook con Husky para ejecutar ESLint antes de ca

INSTALACIÓN:

1. Instalar Husky: `pnpm add -D husky`
2. Inicializar Husky: `pnpm exec husky init`
3. Esto crea carpeta .husky/ con estructura básica

CONFIGURACIÓN PRE-COMMIT:

- Archivo: .husky/pre-commit
- Shebang: `#!/usr/bin/env sh` (requerido para ejecutar script)
- Comandos:
 1. `echo "🔍 Running Quality Gates (pre-commit)..."`
 2. `echo "📝 Checking code style..."`
 3. `pnpm run lint || { echo "❌ ESLint failed!"; exit 1; }`
 4. `echo "✅ Pre-commit gates passed!"`

PERMISOS:

- Ejecutar: `chmod +x .husky/pre-commit` (hace script ejecutable)

PRUEBA:

1. Agregar código con lint error: `const unused = 'test'` (variable no usada)
2. Intentar commit: `git add . && git commit -m "test"`
3. Verificar: Hook debe BLOQUEAR commit mostrando errores ESLint

VALIDACIÓN: Commit debe fallar con mensaje "❌ ESLint failed!" hasta arregl

Aprende: Pre-commit hooks shift-left quality: detectan

problemas ANTES de commit

Ejercicio 2: Pre-push Coverage Gate

Prompt:

Actúa como un ingeniero de calidad configurando coverage gates en pre-push

CONTEXT0: Pre-push hook ejecuta ANTES de git push (checks más lentos: tests coverage, build). Pre-commit = checks rápidos (90s), pre-push = checks comprensivos (2-5 min). Coverage thresholds en vitest.config.ts definen % mínimo requerido. Gate benefit: detecta coverage bajo ANTES de CI (ahorra t de CI + feedback loop más rápido). Exit code: comando retorna 0 (success) o (failure). || operator: ejecuta segundo comando si primero falla. Shift-left testing: mover validación lo más temprano posible en el ciclo de desarrollo

TAREA: Agrega pre-push hook que bloquee push si coverage < 80%.

CONFIGURACIÓN PRE-PUSH:

- Archivo: .husky/pre-push
- Shebang: #!/usr/bin/env sh
- Comandos:
 1. `echo "🚀 Running Quality Gates (pre-push)..."`
 2. `echo "📊 Checking test coverage..."`
 3. `pnpm run test:coverage || { echo "❌ Coverage not met!"; exit 1; }`
 4. `echo "✅ Pre-push gates passed!"`

PERMISOS:

- Ejecutar: `chmod +x .husky/pre-push`

THRESHOLD CONFIGURATION:

- Archivo: vitest.config.ts
- Sección: test.coverage.thresholds
- Configurar:
 - statements: 80 (80% de líneas ejecutadas)
 - branches: 80 (80% de caminos **if/else** cubiertos)

PRUEBA:

1. Comentar tests para bajar coverage: `// it('should...', () => {})`
2. Verificar coverage actual: `pnpm run test:coverage` (debe mostrar < 80%)
3. Intentar push: `git push`
4. Verificar: Hook debe BLOQUEAR push mostrando coverage insuficiente

VALIDACIÓN: Push debe fallar con `❌ Coverage not met!` hasta alcanzar 80%

Aprende: Pre-push coverage gates previenen merges sin tests adecuados

Puntos Clave

1. **Quality Gates (puertas de calidad):** Bloquean código malo automáticamente

2. **Pre-commit (pre-confirmación):** Checks rápidos (lint, tests, build)
3. **Pre-push (pre-empuje):** Checks lentos (coverage, E2E)
4. **Setup Simple:** 3 comandos, 5 minutos
5. **ROI (retorno de inversión) Masivo:** 20-30x en primera semana
6. **Aprendizaje:** Developers aprenden best practices (mejores prácticas)
7. **Prevención:** 90% reducción de bugs